**Colegiul Național „Grigore Moisil”**
București

# Joc video – Dungeon Jumper

Profesor coordonator:

Valentina Preda

Elev:

Crânguș Călin

**Clasa: 12A**

**Mai, 2025**

# Cuprins:

1. De ce Godot? ..... pg. 2
2. Surse de inspirație ..... pg. 3
3. Prezentarea jocului ..... pg. 4
4. Implementarea deplasării jucătorului ..... pg. 5
5. Sistemul de nivele ..... pg. 6
6. Estetică și interactivitate ..... pg. 7
7. Sisteme de control al versiunilor ..... pg. 8

# De ce Godot?




Godot Game Engine este un motor de jocuri puternic, fiind folosit atât de dezvoltatori independenți, cât și de companii mari.

Fiind un program de tip „Free and Open Source Software” (FOSS), acesta poate fi folosit fară nici un fel de taxă, utilizatorii având chiar și libertatea de a modifica și adapta codul motorului pentru fiecare proiect în parte, în funcție de ce au nevoie.

Acesta folosește un limbaj de programare propriu (GDScript), cu sintaxă similară cu Python, dar poate folosi și C# sau C++ pentru situații în care este nevoie de cât mai multă eficiență

Jocuri populare făcute în Godot:

- Brotato
- Buckshot Roulette
- Sonic Colors Ultimate (Remaster)

# Surse de inspirație

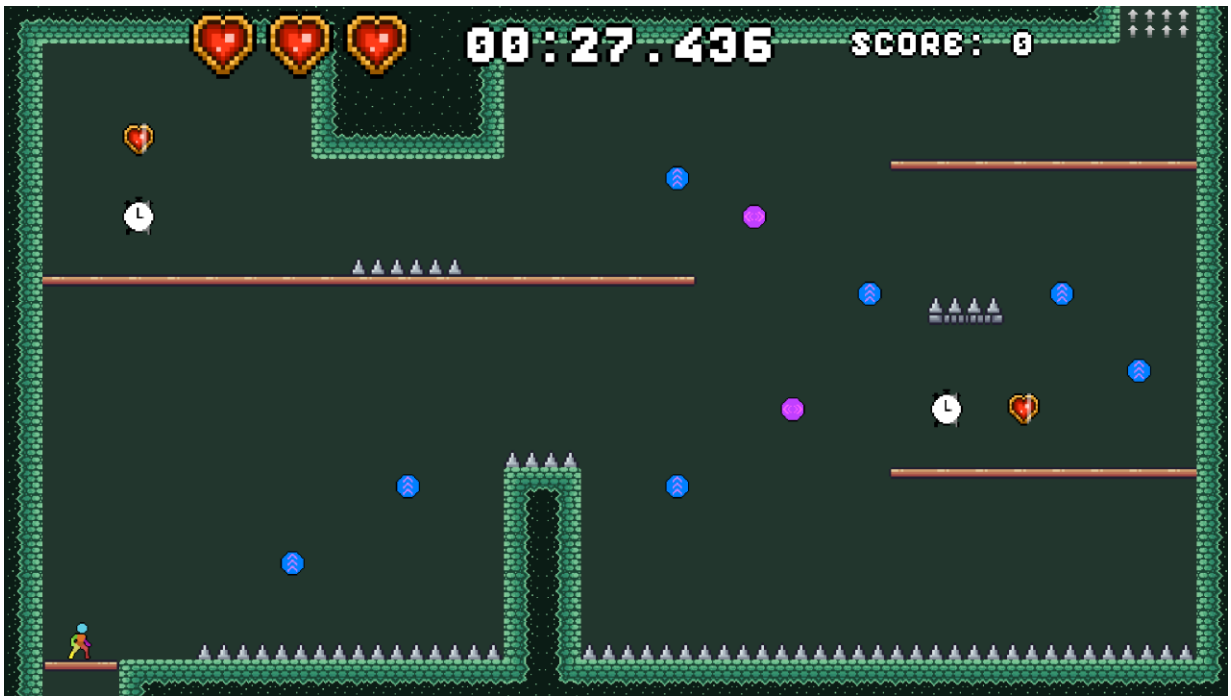



Sursa principală de inspirație pentru acest proiect a fost renumitul joc platformer „Celeste”. Acest joc îmbină o poveste emoționantă cu un set de mișcări precise ce permit un stil satisfăcător de platforming.

Nivelele jocului oferă o provocare corectă și satisfăcătoare, punând accentul pe mișcarea precisă și reacții rapide. Mișcarea definitorie a jocului este „Dash”-ul, cu care jucătorul se poate propulsa într-o direcție, dar care nu poate fi folosit mai mult de odată în aer.

Această dinamică a fost sursa principală de inspirație pentru jocul meu, nivelele jocului fiind construite în jurul acestei mișcări pentru a oferi aceeași senzație de provocare.

# Prezentarea jocului




Scopul jocului este să ajungi în colțul opus al nivelului cât mai repede, evitând țepii. Jucătorul are trei vieți, care pot fi reîncărcate cu ajutorul pickup-urilor în formă de inimă.

Pentru a încuraja parcurgerea rapidă a nivelelor, există un timer ce, odată ajuns la 0, oprește jocul. Acest timer poate fi extins cu ajutorul pickup-urilor în formă de ceas. Scopul este ca un jucător experimentat să poată parcurge nivelele fără a rămâne fără timp, dacă se mișcă optim.

Totuși, pentru a evita situații în care jocul merge la nesfârșit, timer-ul scade din ce în ce mai repede cu cât jucătorul parcurge mai multe nivele.

# Implementarea deplasării jucătorului

```
48      do_jump_logic(delta)
49
50      do_gravity(delta)
51
52      if is_on_floor():
53          air_dashes = 1
54
55      do_crouch_and_slide_logic()
56
57      do_left_right_movement(input_direction)
58
59      do_dash(input_direction, delta)
60
61      cap_speed() # ------------------------------------------------------
```

Sistemul de deplasare a jucătorului este implementat în 7 pași, toți esențiali pentru a asigura mișcarea precisă și corectă

1. Logica de sărit
2. Aplicarea gravitației
3. Resetarea „dash”-urilor pe pământ
4. Logica de ghemuire și alunecare
5. Mișcarea stâga-dreapta
6. Aplicarea „dash”-urilor
7. Limitarea vitezei

# Sistemul de nivele

```gdscript
func start_stage_transition() -> void:
    current_stage = get_node("CurrentStage")
    transition_time_left = transition_time
    transitioning = true
    player.movement_enabled = false
    player_start_position = player.position
    player_target_position = initial_player_position_left if current_stage_direction == -1 else initial_player_position_right
    pause_timer.emit()
    increment_score.emit()

func do_transition(delta: float) -> void:
    transition_time_left -= delta
    if transition_time_left <= 0: return
    var transition_progress_linear: float = 1 - (transition_time_left / transition_time)
    var transition_progress_smooth: float = smootherstep(transition_progress_linear)
    current_stage.position.y = 0 + game_size.y * transition_progress_smooth
    next_stage.position.y = - game_size.y + game_size.y * transition_progress_smooth
    player.position = player_start_position * (1 - transition_progress_smooth) + player_target_position * (transition_progress_smooth)

func smootherstep(x: float) -> float:
    if x <= 0: return 0
    if x >= 1: return 1
    return 6 * (x ** 5) - 15 * (x ** 4) + 10 * (x ** 3) # smootherstep function from https://en.wikipedia.org/wiki/Smoothstep

signal resume_timer()
func end_transition() -> void:
    current_stage.name = "OldStage"
    current_stage.queue_free()
    next_stage.name = "CurrentStage"
    (next_stage.get_node("StageEnd") as Area2D).area_entered.connect(change_stage)
    transition_time_left = 0
    transitioning = false
    current_stage_direction = - current_stage_direction
    player.checkpoint = player.position
    player.movement_enabled = true
    resume_timer.emit()
```

Sistemul de schimbare a nivelelor funcționează prin a alege noul nivel care va fi generat din listă, apoi creerea sa deasupra nivelului curent. Pentru a da iluzia că jucătorul urcă la următorul nivel, cele două nivele sunt mișcate în jos în sincron, iar, la final, nivelul vechi este șters. Acest proces are loc în 3 pași:

1. **Crearea noului nivel**: noul nivel este creat odată ce jucătorul îl termină pe cel curent, și este oglindit în funcție de poziția de început a jucătorului. În plus, timer-ul și mișcarea jucătorului sunt puse pe pauză în timpul tranziției
2. **Mișcarea nivelelor**: cele două nivele sunt mișcare în jos, folosing funcția „smootherstep” pentru a asigura fluiditatea tranziției. De asemenea, jucătorul este poziționat exact în centrul platformei de început, folosind aceeași funcție de atenuare
3. **Finalul tranziției**: nivelul vechi este șters, iar jucătorul și timer-ul sunt dezghețate

# Estetică și interactivitate



Estetica și interactivitatea („Look and feel”) este unul dintre cele mai importate aspecte ale unui joc. Este ceea ce face jocul să fie satisfăcător de jucat, un element extrem de important în special pentru astfel de jocuri bazate pe viteză și precizie.

Elementele importante ce îmbunătățesc „Look and feel”-ul jocului sunt:

1. Grafica specifică „dash”-ului (ilustrat în imagine)
2. Efecte sonore pentru acțiunile posibile din joc (săritura, „dash”-ul, și obținerea „pickup”-urilor)
3. Zguduirea camerei când jucătorul moare pentru a crește impactul

# Sisteme de control al versiunilor




Git este un system de control al versiunilor esențial în dezvoltarea oricărui software, acesta permițând gestionarea eficientă a diferitelor versiuni de cod, dar și pentru a facilita colaboratea mai multori dezvoltatori în cadrul unui singur proiect.

Git este rapid, sigur și open source, fiind standardul folosit în industria de dezvoltare software. Acesta este integrat cu platforme precum GitHub și GitLab, ce permit stocarea în cloud a proiectelor Git, și gestionarea acestora printr-o interfață web.

| | | |
|---|---|---|
| .vscode | Initial commit | last month |
| animations | Added Health, Death, and Camera shake | 4 days ago |
| fonts | added score and a game over screen | 4 days ago |
| scenes | Added "about" page | 1 hour ago |
| scripts | Added "about" page | 1 hour ago |
| shaders | added shine shader to pickups | 3 days ago |
| sounds | Added main menu and tutorial, aka finished game loop | 4 days ago |
| sprites | Added "about" page | 1 hour ago |
| tilesets | Added health pickup | 4 days ago |
| .editorconfig | Initial commit | last month |
| .gitattributes | Initial commit | last month |
| .gitignore | Added gitignore | 23 minutes ago |
| export_presets.cfg | Added gitignore | 23 minutes ago |
| icon.svg | Initial commit | last month |
| icon.svg.import | Initial commit | last month |
| licences.txt | Added Health, Death, and Camera shake | 4 days ago |
| project.godot | Added a time multiplier. The more score you have, the faster... | 3 days ago |